

Conversational AI Chatbot

Architecture & Project Documentation

Project Type: Prompt Engineering Mini-Project

Technology Stack: Python, Flask, DeepSeek V4 Flash Free

Deployment: Render (Free Tier)

Live Demo: <https://ai-assistant-vqkp.onrender.com/>

GitHub: <https://github.com/leenarajput534-stack/ai-assistant>

Built with Python + Flask + OpenCode Zen API

Table of Contents

1. Project Overview
2. Objective
3. Technology Stack
4. System Architecture
5. Chat Modes & Prompt Design
6. Data Flow & Conversation Management
7. API Endpoints
8. User Interface Design
9. Feedback Mechanism
10. Deployment Architecture
11. Expected Outcome

1. Project Overview

The Conversational AI Chatbot is a web-based application designed to demonstrate the power of prompt engineering in building context-aware conversational AI systems. The chatbot engages users in natural conversations across multiple modes, provides informative responses and personalized recommendations, and generates creative content such as stories and poems.

This project fulfills the requirements of a Prompt Engineering mini-project by showcasing three distinct conversational modes, each with carefully crafted system prompts that guide the AI's behavior, tone, and response style. The chatbot maintains conversation context across multiple exchanges, creating a coherent and engaging user experience.

2. Objective

The primary objective of this project is to design and develop a conversational AI chatbot that demonstrates effective prompt engineering techniques. Specifically, the project aims to:

- Create a prompt framework for engaging users on various topics with natural conversation flow
- Provide accurate information and personalized recommendations through well-structured prompts
- Generate creative content (stories, poems, etc.) using imaginative prompt designs
- Maintain context and coherence across multi-turn conversations using session memory
- Implement a feedback loop to collect user input and refine prompt quality

3. Technology Stack

Core Technologies

Component	Technology
Programming Language	Python 3.12+
Web Framework	Flask 3.0+
AI Model API	DeepSeek V4 Flash Free via OpenCode Zen
API Client	OpenAI Python SDK (OpenAI-compatible)
WSGI Server	Gunicorn (Production)
Frontend	HTML5, CSS3, JavaScript
Icons	Font Awesome 6
Session Storage	Flask Signed Cookies
Feedback Storage	JSON File (feedback.json)

Why This Stack?

- Flask: Lightweight, easy to set up, perfect for small-to-medium web applications
- OpenAI SDK: Industry-standard client, works with any OpenAI-compatible API including OpenCode Zen
- DeepSeek V4 Flash Free: Zero-cost, high-quality LLM for generating conversational responses
- Gunicorn: Production-grade WSGI server required by Render for Python apps
- Flask Sessions: Built-in, no database needed for conversation memory

4. System Architecture

The application follows a client-server architecture with a clear separation between the frontend and backend. The user interacts with the chatbot through a web interface, which communicates with the Flask backend via RESTful API calls. The backend manages conversation context, interfaces with the DeepSeek AI model, and stores feedback data.

Architecture Diagram



Component Description

1. User Browser (Frontend)

- Renders the chat interface with message bubbles and mode selection
- Sends user messages to the backend via POST `/chat`
- Displays AI responses with typing animation and feedback buttons
- Manages UI state: chat mode, message history display, loading states

2. Flask Backend

- Routes: `/` (index), `/chat` (POST), `/history` (GET), `/reset` (POST), `/feedback` (POST)
- Session management: stores conversation history in Flask signed cookies
- Prompt engineering: applies mode-specific system prompts to guide AI behavior
- Feedback handling: saves thumbs up/down responses to `feedback.json`

3. OpenCode Zen API

- OpenAI-compatible API endpoint: `https://opencode.ai/zen/v1`
- Model: `deepseek-v4-flash-free` (completely free)
- Receives messages array with system prompt + conversation history + user message
- Returns generated text response with context-aware content

5. Chat Modes & Prompt Design

The chatbot features three distinct conversation modes, each with a dedicated system prompt that defines the AI's persona, behavior, and response style. This demonstrates how prompt engineering can dramatically change the AI's output for different use cases.

Mode 1: General Chat

System Prompt

You are a friendly and engaging conversational AI. Chat naturally with the user on any topic. Be warm, helpful, and keep the conversation flowing. Ask follow-up questions to keep things interesting. Respond in 2-4 sentences unless the user asks for more detail.

- Purpose: Casual conversation, fun facts, daily topics
- Tone: Warm, friendly, engaging
- Strategy: Asks follow-up questions to maintain dialogue flow

Mode 2: Information & Recommendations

System Prompt

You are a knowledgeable guide. Provide accurate information and thoughtful recommendations based on the user's questions. When giving recommendations, ask about their preferences first. Support your answers with clear reasoning. Be concise but thorough.

- Purpose: Facts, explanations, personalized recommendations
- Tone: Professional, informative, helpful
- Strategy: Asks about preferences before recommending

Mode 3: Creative Chat

System Prompt

You are a creative and imaginative conversation partner. Tell stories, write poems, crack jokes, and explore wild ideas. Be playful, use vivid language, and encourage the user to be creative too. Make every response entertaining and inspiring.

- Purpose: Storytelling, poems, jokes, imaginative ideas
- Tone: Playful, vivid, inspiring
- Strategy: Uses rich imagery and encourages user creativity

Prompt Engineering Strategy

Each system prompt is designed with three key elements in mind:

- Persona Definition: The first sentence establishes WHO the AI is (friendly guide, knowledgeable expert, creative partner)
- Behavioral Guidelines: Middle sentences define HOW the AI should act (ask follow-ups, give reasoning, be

playful)

- Structural Constraints: Final sentences set boundaries (response length, tone, approach)

6. Data Flow & Conversation Management

Conversation Flow

1. User Input

User types a message in the chat interface and clicks send.

2. API Request

Frontend sends POST /chat with { mode, message } to the Flask backend.

3. Context Assembly

Backend retrieves conversation history from Flask session (up to 20 messages) and prepends the mode's system prompt.

4. AI Processing

The complete messages array is sent to OpenCode Zen API (DeepSeek V4 Flash Free) for response generation.

5. Response Handling

Backend receives the AI response, stores it in session, and returns it to the frontend.

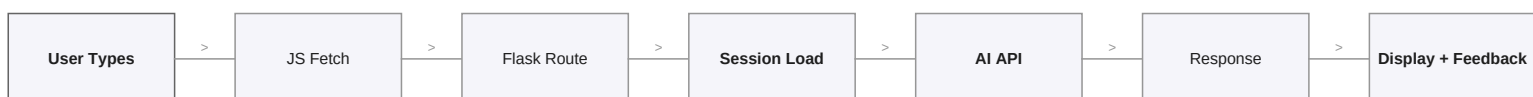
6. Display & Feedback

Frontend displays the response in a chat bubble and shows thumbs up/down buttons for feedback.

Session Memory Management

Conversation context is maintained using Flask's signed cookie-based sessions. The conversation history is stored as a list of { user, bot, mode } objects, with a maximum of 20 exchanges. When building the API request, the last N-1 messages are included to stay within token limits. Users can reset the conversation at any time via the /reset endpoint.

Data Flow Diagram



7. API Endpoints

POST /chat

Send a message and receive an AI-generated response.

```
Request: { "mode": "general", "message": "Hello!" }
Response: { "response": "Hi there! How can I help you today?" }
```

Parameter	Description
mode	Chat mode: general / info / creative
message	User's input text

GET /history

Retrieve the current conversation history.

```
Response: { "history": [
  { "user": "Hello", "bot": "Hi!", "mode": "general" },
  ...
] }
```

POST /reset

Clear the conversation history and start fresh.

```
Response: { "status": "ok" }
```

POST /feedback

Submit user feedback on a specific response.

```
Request: {
  "mode": "general",
  "message": "Tell me a fun fact",
  "response": "Did you know octopuses...",
  "helpful": true
}
Response: { "status": "ok" }
```

8. User Interface Design

The user interface is a single-page web application with a chat-based design. It features a dark theme with gradient backgrounds for a modern, visually appealing experience.

UI Components

- Header: Displays the chat title and AI model information
- Mode Selector: Three toggle buttons to switch between General, Info, and Creative modes
- Chat Area: Scrollable message container with user and bot message bubbles
- Input Area: Text input field with send and reset buttons
- Suggestions: Quick-start suggestion chips for each mode
- Feedback: Thumbs up/down buttons on each bot response
- Typing Indicator: Animated dots while AI generates response

Design Highlights

- Responsive layout - works on mobile, tablet, and desktop
- Dark theme with gradient backgrounds (deep purple to blue)
- Smooth fade-in animations for messages
- Distinct message bubble styles for user (purple gradient) vs bot (translucent)
- Mode-specific accent colors for visual differentiation

9. Feedback Mechanism

A feedback loop is implemented as required by the project specification. After each AI response, users can provide thumbs up or thumbs down feedback. This feedback is stored in a JSON file (feedback.json) with the following structure:

```
{
  "mode": "general",
  "message": "Tell me a fun fact",
  "response": "Did you know...",
  "helpful": true,
  "timestamp": "2026-07-25T13:10:27"
}
```

The collected feedback can be analyzed to identify which prompt modes and styles are most effective, allowing for iterative refinement of the system prompts.

10. Deployment Architecture

The application is deployed on Render's free tier using a production-grade WSGI server.

Live URL

<https://ai-assistant-vqkp.onrender.com/>

Source Code

<https://github.com/leenarajput534-stack/ai-assistant>

Deployment Configuration

Setting	Value
Platform	Render.com (Free Tier)
Build Command	pip install -r requirements.txt
Start Command	gunicorn app:app
Python Version	3.12+
WSGI Server	Gunicorn (sync worker)
Environment Variables	ZEN_API_KEY, SECRET_KEY

Project Structure

```
ai-assistant/  
  app.py          # Flask app with chat logic & session memory  
  templates/  
    index.html    # Chat UI (HTML/CSS/JS)  
  requirements.txt # Python dependencies  
  .env.example    # Environment variable template  
  feedback.json   # User feedback storage  
  .gitignore  
  LICENSE  
  README.md
```

11. Expected Outcome

This project successfully demonstrates the core principles of prompt engineering through a functional, context-aware conversational AI chatbot. The expected outcomes are:

- A set of three carefully crafted system prompts that effectively guide AI behavior across different conversation modes
- A fully functional web-based chatbot with natural conversation flow and context retention
- Demonstration of prompt structure: persona definition, behavioral guidelines, and output constraints
- A feedback loop that captures user input for prompt refinement and quality assessment
- A production-ready application deployable on cloud platforms with minimal configuration

Learning Outcomes

- Understanding how system prompts influence AI tone, style, and content
- Practical experience with OpenAI-compatible APIs and LLM integration
- Building context-aware conversational systems with session management
- Full-stack web development with Flask and modern JavaScript
- Deploying Python web applications to cloud platforms